Subject outline

Lecture topics (by week)

1 – Introduction to quantum computing and maths basics
2 – Single qubit representations and logic operations
3 – Two qubit states and logic gates
4– Multi-qubit states and quantum arithmetic
5 – Basic quantum algorithms
6 – Period finding, cryptography and quantum factoring
7– Shor's algorithm, post-quantum crypto, quantum key distribution
8 – Quantum search algorithms
9 – Grover search applications, optimisation problems
10 – Solving optimisation problems on quantum computers
11– Applications in quantum machine learning
12 – Real quantum computer devices

Assignment schedule:
#1: Hand out in Week 2
#2: Hand out in Week 8

Week 11

Lecture 21

1. Machine Learning
2. Neural Network
3. Support Vector Machine
4. Principal Component Analysis

Lecture 22

1. Quantum Machine Learning
2. Quantum Principal Component Analysis

Practice class 11

Quantum Machine Learning

21.1 Machine Learning

MULT20015

Lecture 21

Machine Learning

What is machine learning?

Machine learning definition by Arthur Samuel from 1959:

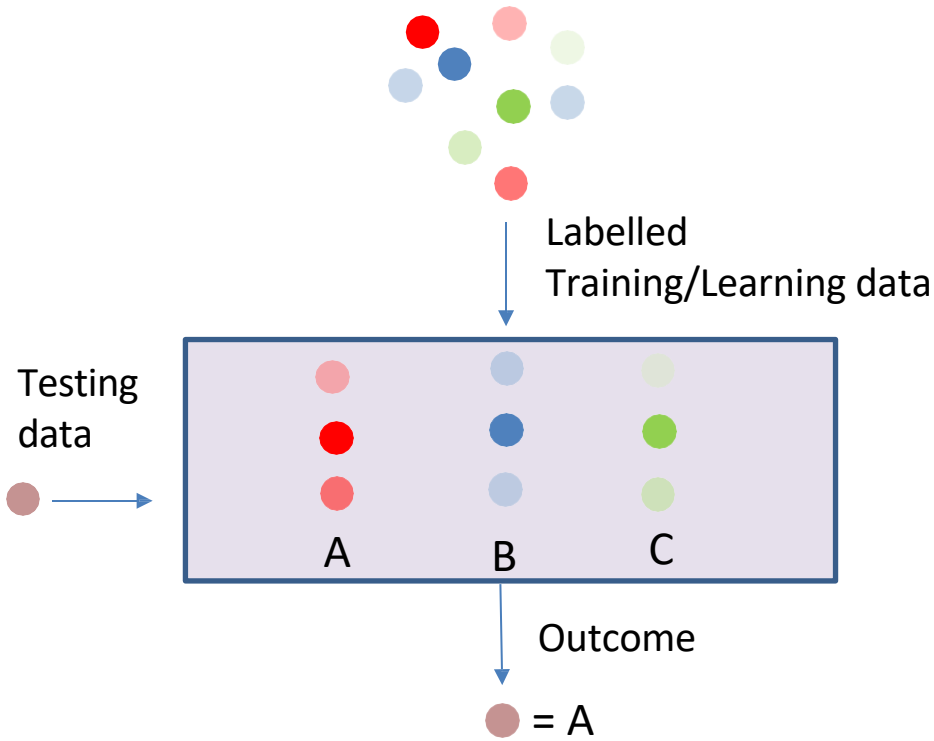
"[Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed."

Machine learning is an integral part of our daily lives

1. Generative AI (ChatGPT4, Midjourney etc.)
2. Programming (GitHub Copilot)
3. Social networking (TikTok, LinkedIn, Facebook etc.)
4. Autonomous cars (Waymo in San Francisco, Baidu in Beijing's Yizhuang district, etc.)
5. Advertising (Google, Netflix, Amazon, etc.)
6. Transcription & chatbot speech recognition (Zoom, Teams, Siri, Customer support, etc.)
7. Email spam filtering & cybersecurity etc.

Major Categories of Machine Learning Techniques

Supervised Machine Learning



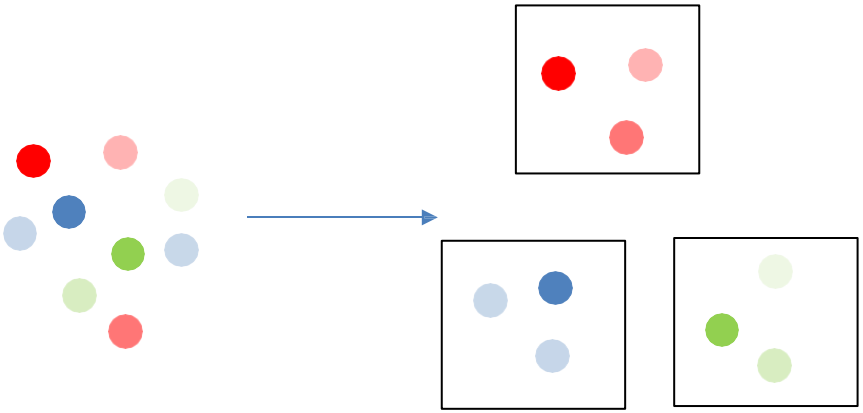
Examples:

Neural Networks/ Convolutional Neural Networks, Deep Learning

Support Vector Machines (SVMs)

Unsupervised Machine Learning

No training data required



Examples:

Principle Component Analysis (PCA)

Clustering

Singular Value Decomposition (SVD)

21.2 Neural Network

MULT20015

Lecture 21

Example 1: Neural Networks

Neural Networks can utilise *supervised* machine learning, typically used for *classification*.

More advanced versions are *convolutional neural networks*, *deep neural networks (such as attention-based transformers).*

Some Applications of Neural Networks:

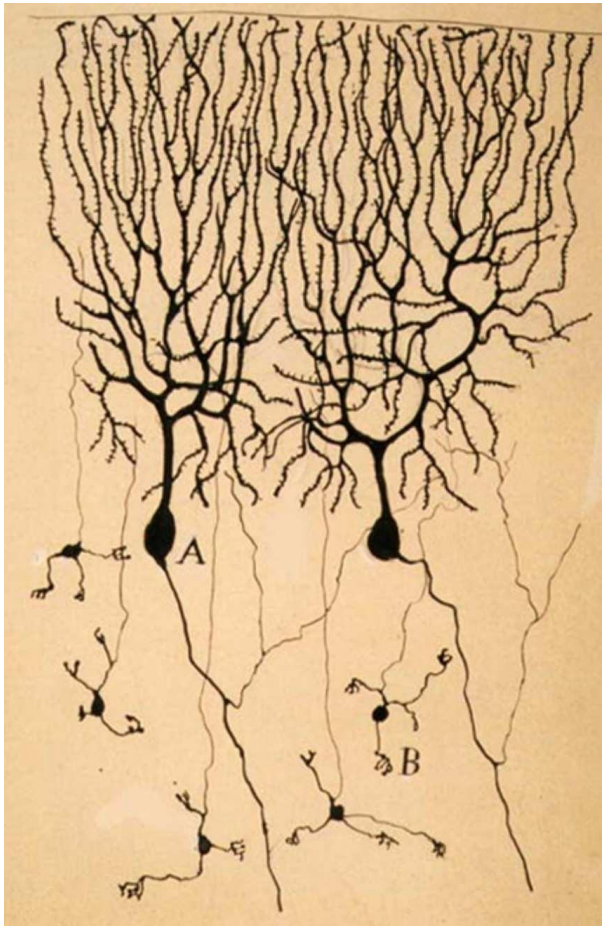
- Recognize images
- Describe images in sentences
- Colorize images
- Translate languages (French to Spanish, etc.)
- Answer questions about a text (ChatGPT etc.)
- Play video games & board games at superhuman level
- Drive a car

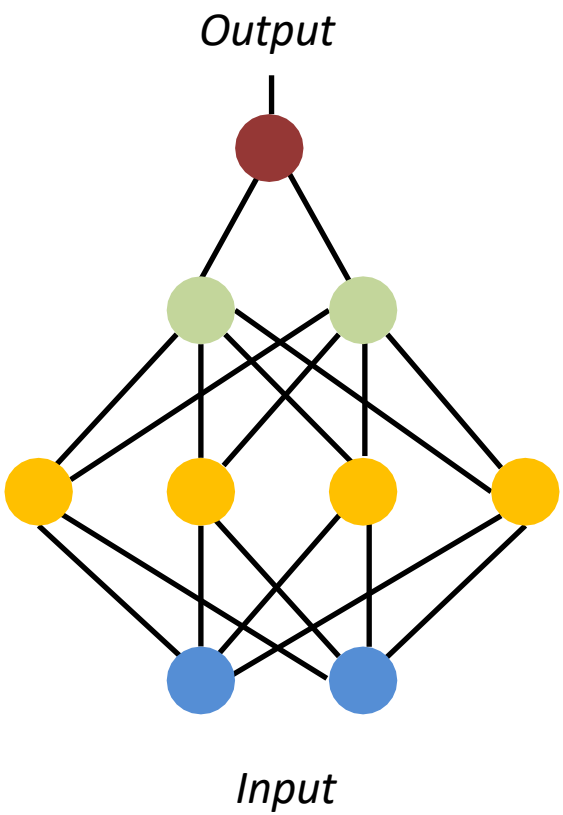
(in physical sciences:)

- Prediction of properties of materials
- Classifying phases of matter
- Representing quantum states

Basic Structure of a Neural Network

Neural Networks were inspired by arrangement and connections of neurons in brain





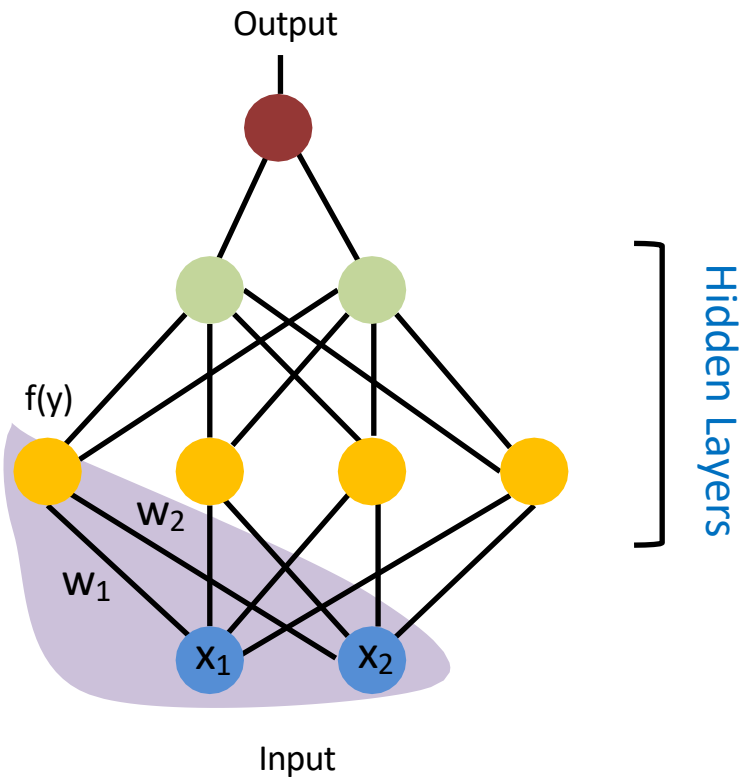
Drawing by Santiago Ramón y Cajal (1899) of neurons in the pigeon cerebellum

Construction of a Neural Network

A Neural Networks are essentially a nonlinear function of multiple input parameters.

The value of two input neurons is: x_1 and x_2

- The weights on the edges connecting input neurons to hidden layers are w_1 and w_2.

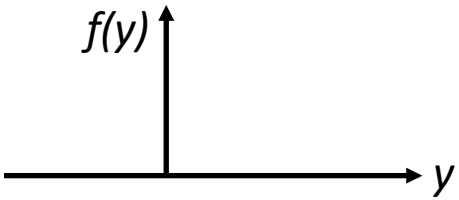


The value at the *hidden layer* neuron is $f(y)$

$y = w_1x_1 + w_2x_2 + b$ (b is *bias* or *offset*)

and

$f(y)$ is a nonlinear function of y.



Working of a Neural Network

A neural network requires a large number of data samples, known as training data. The training data must be *labelled* with the correct classifications.

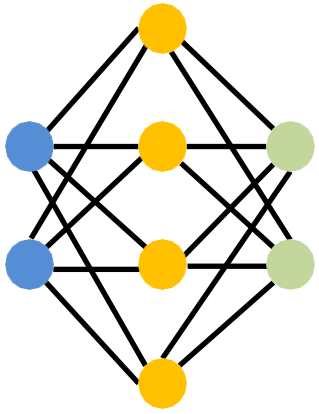
Learning: The neural network is trained using the training data by *back propagation* from known outputs.

Testing: A well-trained neural network is tested with test data to determine its accuracy

Training data

"MNIST" data set (for postal code recognition)
http://yann.lecun.com/exdb/mnist/

Neural Network



Test data

Outputs

21.3 Support Vector Machine

MULT20015

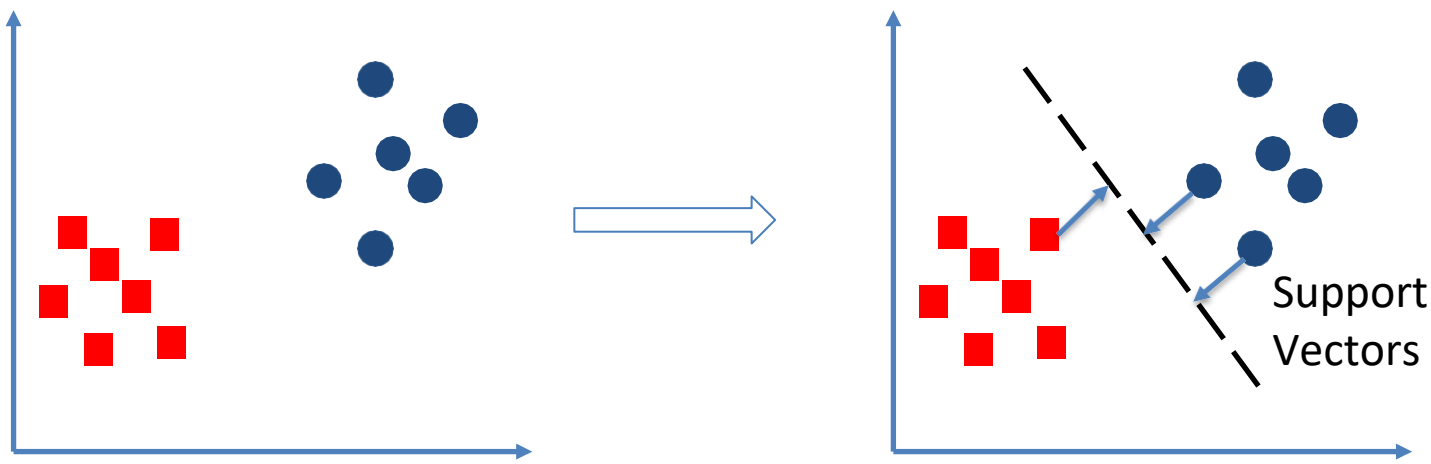
Lecture 21

Example 2: Support Vector Machine

Support Vector Machine (SVM) is a *supervised* machine learning technique.

Let's look at an example of a data set in 2-dimensional plane [n=2]

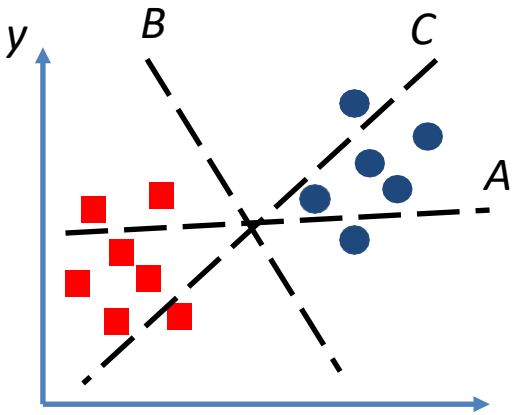
- For n=2, the hyper-plane would be a line,
- for n=3, the hyper-plane would be a plane; and so on…



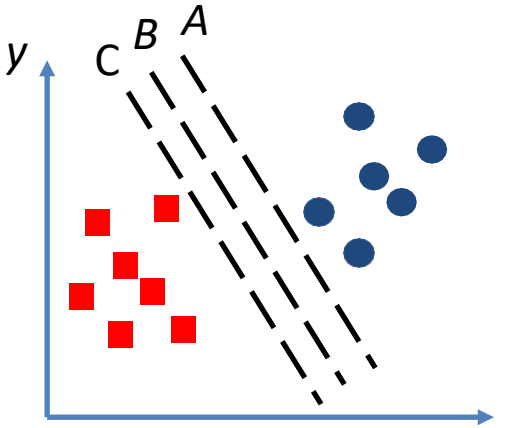
SVM classifier finds a best hyper-plane to classify data entries in two categories.

Best choice for a hyper-plane

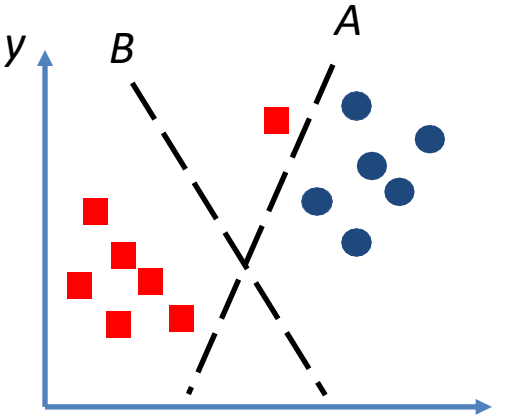
The best choice for a *hyper-plane* depends on data distribution such as margin of error (defined in next slide) and classification error. Best hyper-plane maximizes margin and robustness.



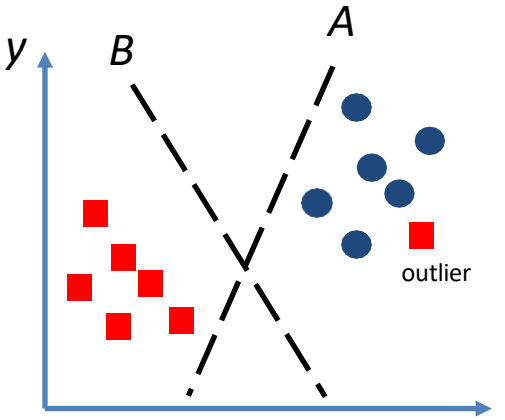
B is the best choice



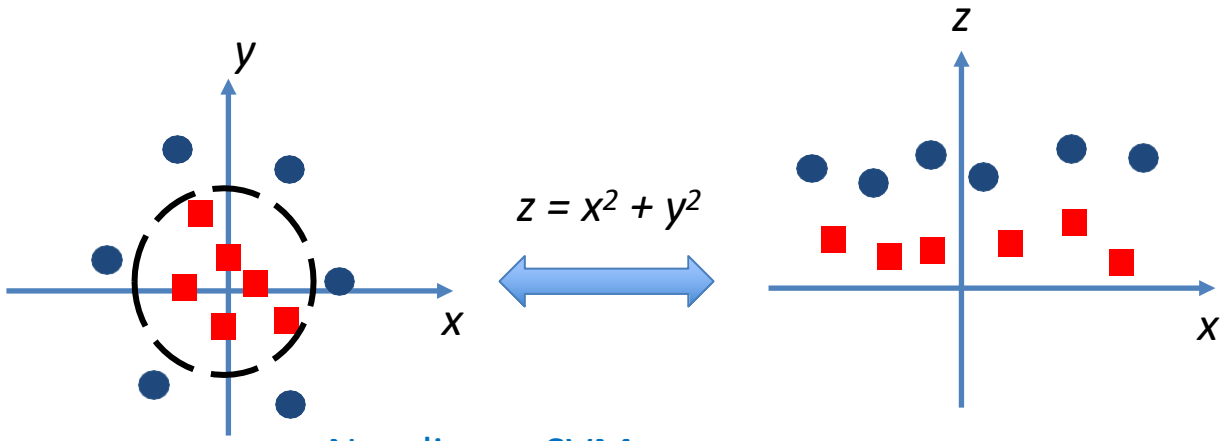
B is the best choice



A is the best choice



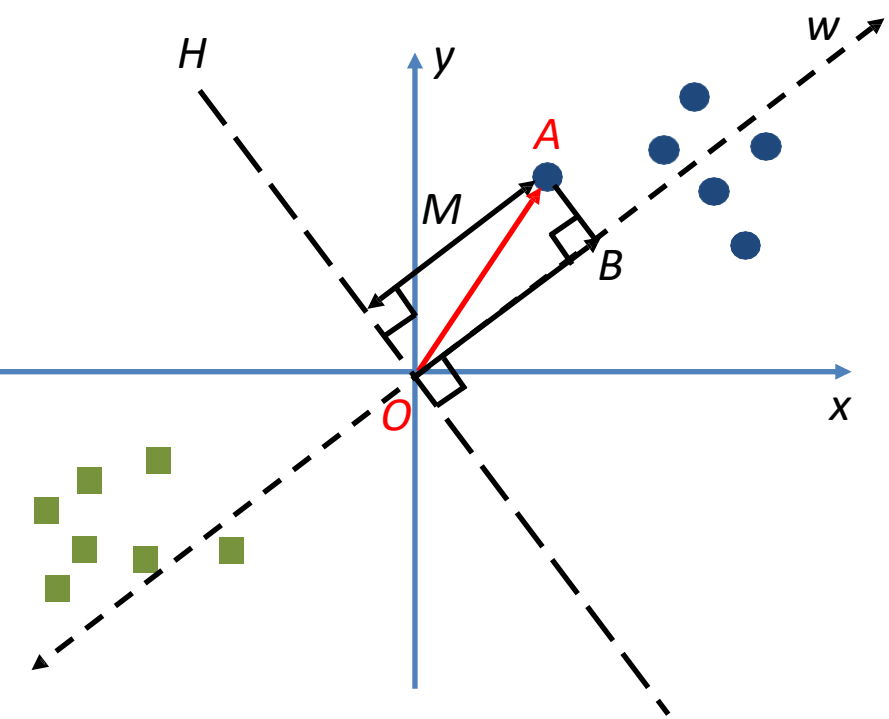
B is the best choice



Non-linear SVM

Calculation of Margin – 2D Example

The Margin, *M*, of a point is the shortest *orthogonal* distance between a the data point and the hyper-plane.



- $\overrightarrow{OA}$ is a *vector* from origin, *O*, to point *A*
- Given equation of a line is: $y=ax+b$ or $y-ax-b=0$

This can be re-written in *vector* form as $w^T z = 0$

(assuming *column* vectors $w \,.\, z = w^T z$)

where $w=\begin{pmatrix} -b \\ -a \\ 1 \end{pmatrix}$ and $z = \begin{pmatrix} 1 \\ x \\ y \end{pmatrix}$

For the hyper-plane *H*: $y=-ax \rightarrow w=\begin{pmatrix} 0 \\ a \\ 1 \end{pmatrix}$, $z = \begin{pmatrix} 1 \\ x \\ y \end{pmatrix}$

Orthogonal projection vector of $\overrightarrow{OA}$ onto *w* is:

$$\vec{P} = (\hat{OB}.\ \overrightarrow{OA})\ \hat{OB}$$

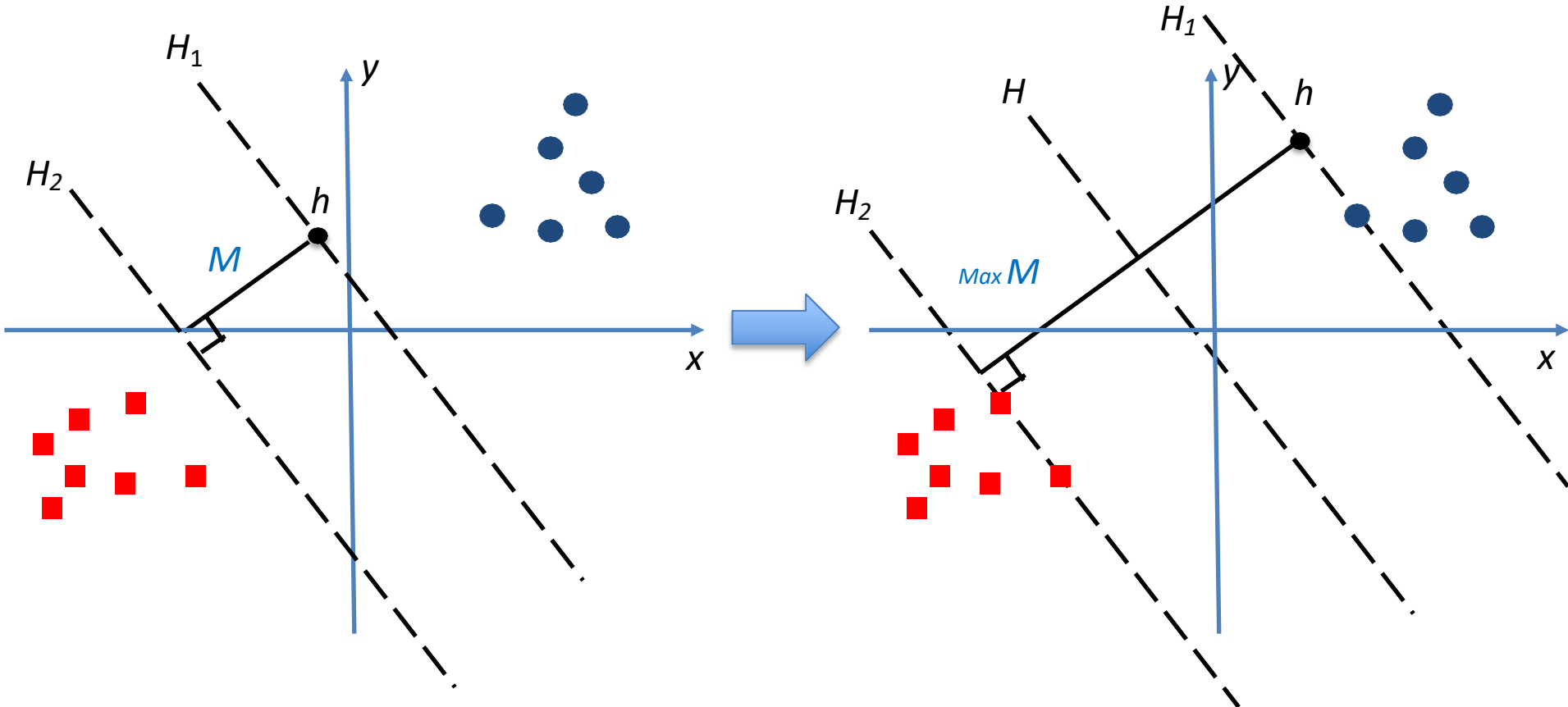
where $\hat{OB} = \dfrac{\overrightarrow{OB}}{\|\overrightarrow{OB}\|} = \dfrac{w}{\|w\|}$

Therefore, $M = \|\vec{P}\|$

Finding an optimal hyper-plane

Finding an optimal hyper-plane involves *maximizing* the margin – i.e. between two hyper-planes (H_1 and H_2) with no data points in between them

Maximize margin M between H_2 and a point h on H_1



21.4 Principal Component Analysis

MULT20015

Lecture 21

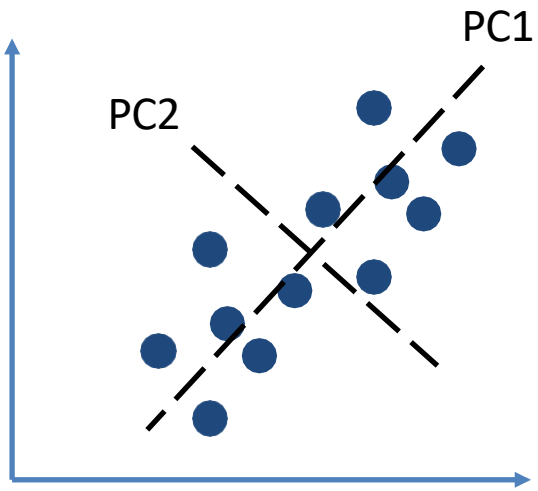
Example 3: Principal Component Analysis

Principle Component Analysis (PCA), as the name indicates, finds the principal component(s) of a data set = the direction in space along which input data has the largest variance.

Main advantage of PCA: Reduction of data dimensionality while preserving patterns and correlations, such that the low dimensional data still contains significant features of the original data.

The working of PCA is divided into following steps:

1. Standardization of the data set
2. Computing the covariance matrix
3. Principal Components: eigenvectors and eigenvalues of the covariance matrix
4. Dimension reduction of the data set



PCA is an *unsupervised* machine learning technique, frequently used to reduce the size of training data, but can be used in its own right

Working of Principle Component Analysis

Standardisation of data is required so values in all dimensions *lie in the same range*.

The data is adjusted using *mean* and *standard deviation* values along each dimension.

Adjusted data = $\frac{Original\ data - mean}{standard\ deviation}$

Working of Principle Component Analysis

Standardisation of data is required so values in all dimensions *lie in the same range*.

The data is adjusted using *mean* and *standard deviation* values along each dimension.

$$\text{Adjusted data} = \frac{Original\ data - mean}{standard\ deviation}$$

Age versus salary Example:

Age (Years)	Salary ($)
45	90000
35	70000
25	35000
40	100000
41	95000
37	83000
50	125000

Age	Salary
0.82	0.87
-0.54	-1.52
-1.90	-5.69
0.14	2.04
0.27	1.46
-0.27	0.03
1.50	5.07

Working of Principle Component Analysis

Computing the covariance matrix: [also recall Lecture 20]

Covariance matrices find correlations and dependencies between dimensions of data.

For two-dimensional data *(X,Y):*

$$Cov(X,Y) = \frac{\sum_{i=1}^{n}(X_i - E[X])(Y_i - E[Y])}{n-1}$$

where *E[X]* and *E[Y]* are the mean (or expected) values of the data along *X* and *Y* dimensions.

And the *covariance matrix* will be: $\begin{pmatrix} Cov(X,X) & Cov(X,Y) \\ Cov(Y,X) & Cov(Y,Y) \end{pmatrix}$

For example *X* could be *Age* and *Y* could be the *Salary* as in the data.

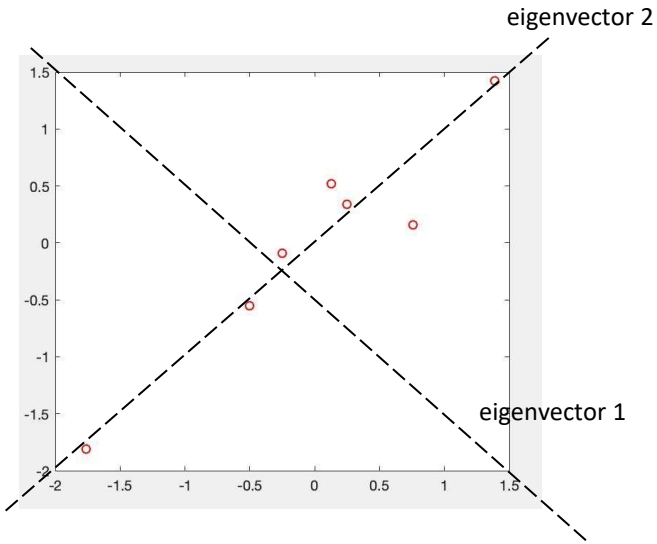
From our example, the covariance matrix is: $\begin{pmatrix} 1 & 0.95 \\ 0.95 & 1 \end{pmatrix}$

The sign of covariance indicates the dependence type between two variables:

- A *positive Cov(X,Y)* value indicates an increase in *X* will lead to increase in *Y*.
- A *negative Cov(X,Y)* value indicates an increase in *X* will lead to decrease in *Y* (*or vice versa*).
- A zero *Cov(X,Y)* value indicates no dependence between *X* and *Y*.

Working of Principle Component Analysis

Computing principal components of data involves finding eigenvalues and eigenvectors of the *covariance matrix*.



Note: For n-dimensional data:

- The covariance matrix *dimension* is *n*n*
- Therefore, *n* eigenvalues/vectors are computed

For our example data:

- Eigenvalues are: *0.0457* and *1.9543*
- Eigenvectors are: $(-0.7071, 0.7071)^T$ and $(0.7071, 0.7071)^T$

To find r principal components, *n* eigenvalues can be sorted in *descending* order *largest* r values are stored.

The *r* eigenvectors correspond to the *r* eigenvalues from the *principal components* of the data.

Note: if the ignored eigenvalues are very small, the information lost is *insignificant*.

- In example, we can find the principal component by keeping one eigenvalue 1.9543

Working of Principle Component Analysis

Dimension reduction of the data set

The final step in PCA is to reduce the *n-dimensional* data to *r-dimensional data* by using *r* principal components obtained, by retaining the highest r number of eigenvalues.

This can be done by multiplying *n-dimensional* data set with *r-dimensional* principal component matrix (also known as *feature matrix*):

- Final data = principal component *x* adjusted data
- In our case, we keep the eigenvector for the larger eigen value: $(0.7071, 0.7071)^T$

Working of Principle Component Analysis

Dimension reduction of the data set

The final step in PCA is to reduce the *n-dimensional* data to *r-dimensional data* by using *r* principal components obtained, by retaining the highest r number of eigenvalues.

This can be done by multiplying *n-dimensional* data set with *r-dimensional* principal component matrix (also known as *feature matrix*) using dot products:

- Final data = principal component *x* adjusted data
- In our case, we keep the eigenvector for the larger eigen value: $(0.7071, 0.7071)^T$

So, the final data in our example will be:

Age	Salary
0.82	0.87
-0.54	-1.52
-1.90	-5.69
0.14	2.04
0.27	1.46
-0.27	0.03
1.50	5.07

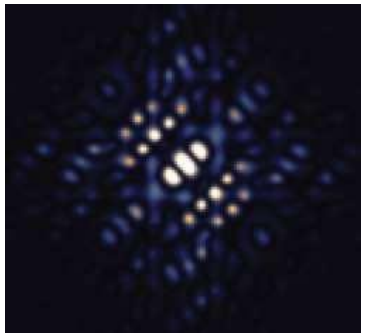
$x\ (0.7071, 0.7071)^T =$

Final Data
1.20
1.45
-5.37
1.53
1.22
-0.16
4.65

PCA for Image Compression

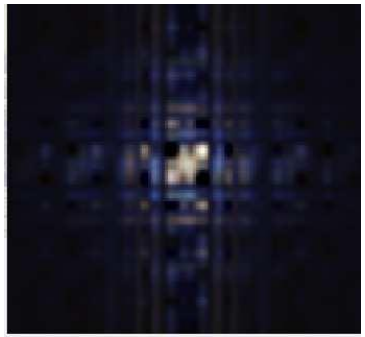
Image compression example: 2-dimensional image data is used to compute covariance matrix whose eigen values represents principal components (we don't go into detail).

Original (uncompressed) image

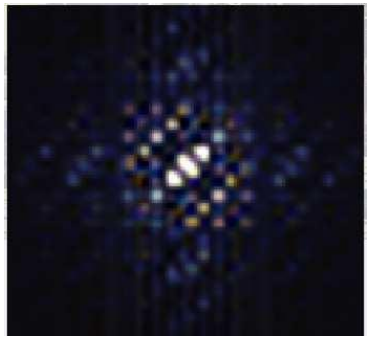


Reduced Images based on PCA

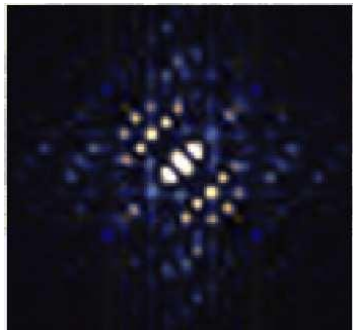
2 principal components (PCAs)

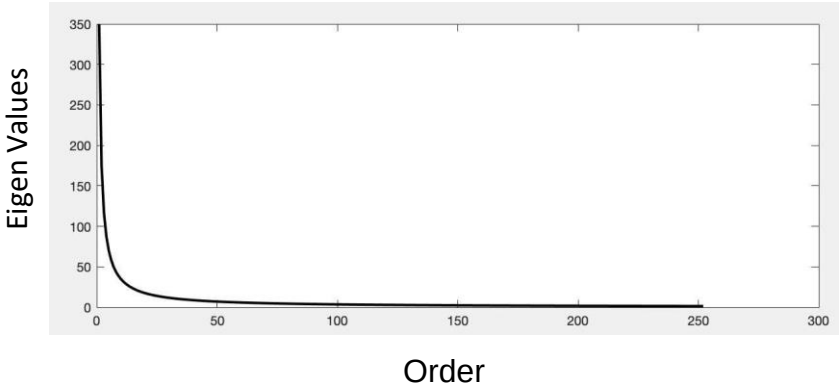


5 PCAs

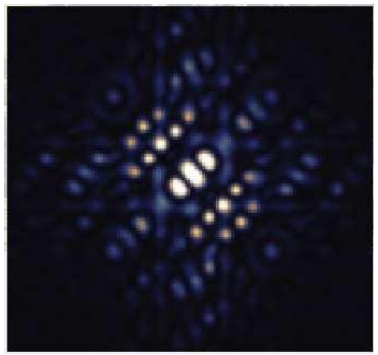


10 PCAs

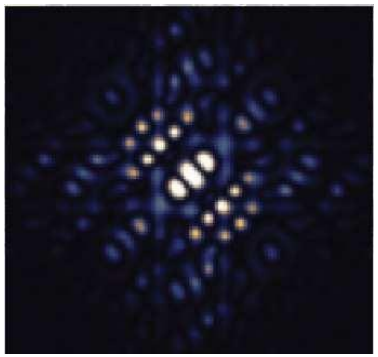




20 PCAs



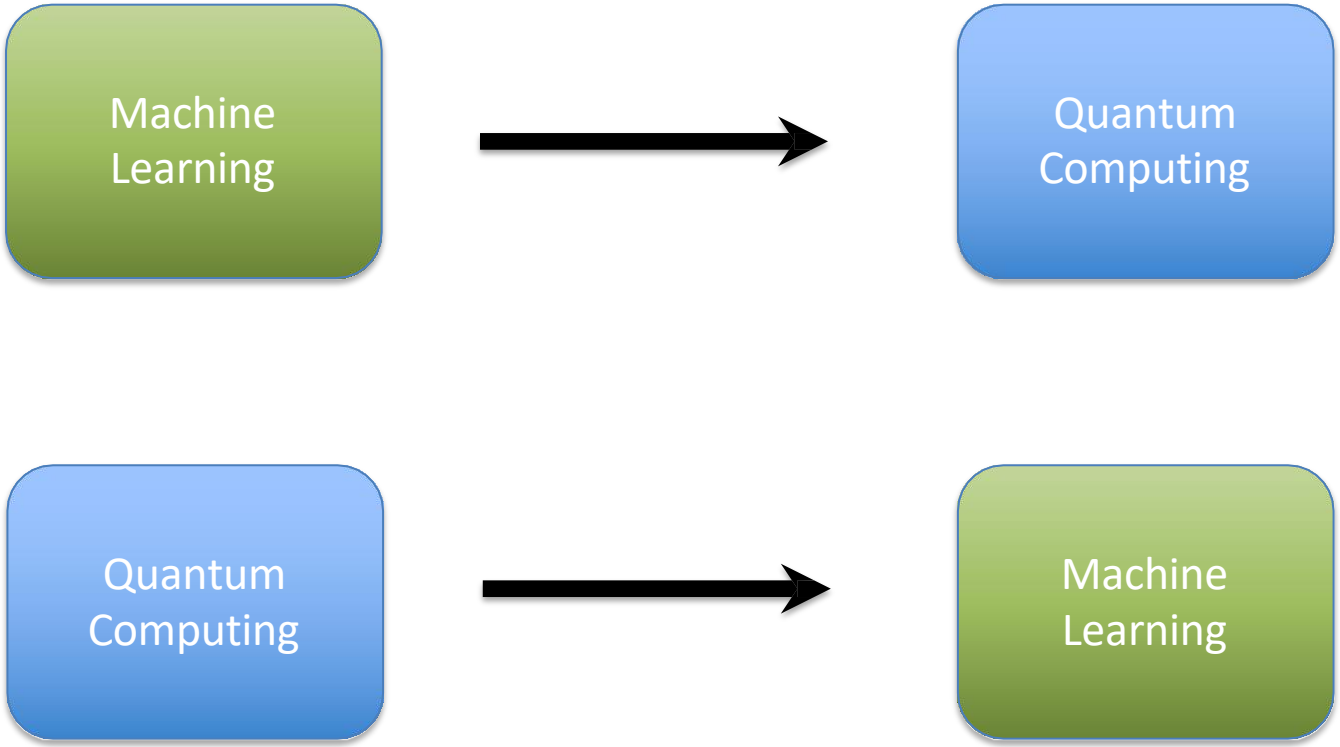
30 PCAs



PCA can *dramatically* reduce the image size by keeping only 50 corresponding eigenvectors.

Intersection of Quantum Computing and Machine Learning

- The intersection of machine learning and quantum computing has seen rapid progress in recent years.
- There are two clear directions in this cross-disciplinary research we will investigate in the next lecture.



Week 11

Lecture 21

1. Machine Learning
2. Neural Network
3. Support Vector Machine
4. Principal Component Analysis

Lecture 22

1. Quantum Machine Learning
2. Quantum Support Vector Machine
3. Quantum Principal Component Analysis

Practice class 11

Quantum Machine Learning